

Code review — solar-sync-broker-api

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 5 — 2026-09-09 — sha cc74f0c

Branch: `master` (identic cu `epic-6`). Atenție: `feat/epic-5` și `feat/fundatie` sunt **strămoși** ai lui `master`, nu linii de lucru separate — tot ce ai scris e deja în `master`.

Subiect: EPIC-6 — endpoint-ul de comandă (`POST/GET /battery/commands`), entitatea `CommandExecution`, plus slice-ul `telemetry` început pe `feat/fundatie`.

Verificat prin rulare, în clonă separată, pe H2 — codul tău neatins:

```
mvn -q -DskipTests compile → BUILD SUCCESS (codul de producție compilează)
mvn -q test-compile        → BUILD FAILURE (testele NU compilează – vezi B1)
```

După ce am reparat în *clona mea* linia care rupe compilarea, am pornit aplicația și am exersat efectiv fiecare cale semnalată mai jos. Fiecare 🚩 are dedesubt output-ul real, nu o presupunere.

Legendă severitate

- 🚩 Critice (B1, B2...) — nu compilează, bug-uri, securitate, date greșite
- 🟡 Importante (M1, M2...) — design, model de domeniu, pattern greșit
- 🟢 Cleanups (C1, C2...) — stil, naming

Ce e bine ✅

- BATTERY_VIEW s-a reparat.** În pass 3 token-ul conținea `battery:view` iar controllerul cerea `BATTERY_VIEW`. Acum `UserPermissions` e un enum simplu, `User.getAuthorities()` mapează `p.name()`, iar `hasAuthority('BATTERY_VIEW')` se potrivește. Ai eliminat sursa de nepotrivire, nu doar un string.
- Ai închis slice-ul până la HTTP.** `POST /api/v1/houses/{houseId}/battery/commands` există, întoarce `201 CREATED`, cu `@Valid`, `Principal` și un DTO de răspuns. E exact mișcarea pe care ți-am cerut-o la B2 pass 4 — cererea ajunge acum până în baza de date.
- Comenzile se persistă.** `CommandExecution` + `CommandExecutionRepository` + mapper static — ai un audit trail (cine, ce, când). Corect ca idee de domeniu.
- Mapper static, fără stare**, în stilul lui `BatteryMapper` — consecvent cu restul proiectului.
- `permissionsForType()` — ideea de a deriva permisiunile din tipul de user, într-un singur loc, e corectă. Problema e ce conține setul (B3), nu unde stă.

B1. Testele nu compilează — `mvn test` nu rulează NIMIC

`src/test/java/ro/mycode/solarsyncbroker/unitTests/serviceTests/BatteryCommandServiceTests.java:35`

CUM E ACUM:

```
mvn -q test-compile
[ERROR] BatteryCommandServiceTests.java:[35,19] constructor BatteryCommandServiceImpl
cannot be applied to given types;
required: BatteryRepository, BatteryCommandValidator, CommandExecutionRepository, HouseRepository
found:    BatteryRepository, BatteryCommandValidator
reason: actual and formal argument lists differ in length
```

Ai adăugat două dependențe noi în constructorul lui `BatteryCommandServiceImpl` (`CommandExecutionRepository`, `HouseRepository`) și n-ai actualizat testul care îl construiește manual.

DE CE: Maven compilează `src/test` ca un modul separat, dar **într-o singură unitate** — o clasă de test care nu compilează oprește `test-compile`, deci nicio clasă de test nu ajunge să ruleze. Nu pierzi un test, pierzi suita. E o treaptă mai jos decât problema din pass 4: acolo 16 teste erau *sărite* în tăcere, aici **toate cele 40+** nu mai există pentru build.

E și motivul pentru care n-ai simțit nimic: ai rulat aplicația, nu testele. Un test unitar care instanțiază serviciul cu `new` e cuplat la semnătura constructorului — orice dependență nouă în producție rupe testul. Cu `@InjectMocks` în loc de `new`, Mockito ar fi completat singur argumentele noi și testul ar fi rămas verde.

FIX: `new BatteryCommandServiceImpl(repository, validator, commandExecutionRepository, houseRepository)` cu două `@Mock` în plus — sau, mai bine, `@InjectMocks BatteryCommandServiceImpl service;`.

B2. `GET /battery` întoarce 500 pentru ORICE user care nu e ADMIN — inclusiv proprietarul

`BatteryQueryServiceImpl.java:31`

Reprodus, proprietarul cerându-și propria baterie:

```
PROBE10 OWNER isi vede bateria -> status 500
| LazyInitializationException: Could not initialize proxy [House#1] - no session
```

```
if (user.getUserType() != UserType.ADMIN && !battery.getHouse().getOwner().getId().equals(user.getId())) {
```

DE CE: `Battery.house` e `@OneToOne(fetch = LAZY)` și `House.owner` e `@ManyToOne(fetch = LAZY)`. Repository-ul întoarce bateria cu un **proxy** în locul casei — un obiect gol care știe doar id-ul și care se umple *la prima atingere a unui alt câmp*, dar numai dacă sesiunea Hibernate mai e deschisă. Metoda ta nu e `@Transactional`, iar în `application.yml` ai `open-in-view: false`, deci sesiunea se închide când iese din repository. Când ajungi la `.getOwner()`, proxy-ul nu mai are de unde încărca → excepție.

De ce ADMIN scapă: Java evaluează `&&` de la stânga la dreapta și se oprește la primul `false`. Pentru un ADMIN prima condiție e falsă, deci `.getHouse()` nu se atinge niciodată. Bug-ul e invizibil exact pentru contul cu care testezi tu.

Și de ce apare **abia acum**: în pass 3 endpoint-ul returna 403 pentru toți (nepotrivire de authority). Ai reparat authority-ul, cererea a ajuns pentru prima dată în serviciu — și a lovit stratul următor. Asta e normal și e semn de progres; important e că fiecare strat nou trebuie exersat, nu presupus.

FIX: `@Transactional(readOnly = true)` pe metodă, **sau** — mai bine — o interogare care aduce datele necesare dintr-un foc: `findByHouseIdWithOwner(...)` cu `JOIN FETCH b.house h JOIN FETCH h.owner`.

B3. `BATTERY_COMMAND` și `BATTERY_CONFIG` nu se acordă nimănui — endpoint-ul nou e 403 pentru toată lumea

`AuthServiceImpl.java:99-112` vs `BatteryController.java:41,48,59`

Reprodus:

```
PROBE1 admin authorities -> [HOUSE_MANAGE, BATTERY_MANAGE, HOUSE_VIEW, BATTERY_VIEW]
PROBE1 are BATTERY_COMMAND? -> false
PROBE1 are BATTERY_CONFIG? -> false
PROBE2 POST /battery/commands ca ADMIN real -> status 403
PROBE8 PATCH /battery/configuration ca ADMIN real -> status 403
```

`UserPermissions` are 6 valori. `permissionsForType()` acordă 4 (`HOUSE_VIEW`, `HOUSE_MANAGE`, `BATTERY_VIEW`, `BATTERY_MANAGE`). Controllerul cere `BATTERY_CONFIG` și `BATTERY_COMMAND` — exact cele două care nu se acordă niciodată. `BATTERY_MANAGE` se acordă, dar nu-l cere nimeni.

DE CE: `@PreAuthorize("hasAuthority('X')")` nu întreabă „există X în enum?”, ci „conține token-ul acestui user string-ul exact X?”. Enum-ul e doar vocabularul; **lista efectivă de permisiuni se decide în `permissionsForType()` și se îngheață în token la login**. Sunt două locuri care trebuie să spună același lucru, și n-ai nicio verificare automată că o fac.

Ăsta e al treilea pass consecutiv în care slice-ul de baterie e inaccesibil dintr-un motiv de authority, de fiecare dată altul: pass 3 = nume diferit (`battery:view` vs `BATTERY_VIEW`), pass 5 = nume corect dar neacordat. Ai reparat **linia** semnalată, nu **regula**: „orice authority cerută de un controller trebuie să fie acordată cuiva”. Un singur test `MockMvc` pe fiecare endpoint, cu token-ul real al unui user real, ar fi prins toate cele trei runde.

FIX: adaugă `BATTERY_CONFIG` și `BATTERY_COMMAND` în setul ADMIN (și decide dacă un USER obișnuit are voie să-și comande propria baterie — probabil da). Apoi șterge `BATTERY_MANAGE`, care nu e cerut nicăieri.

B4. `executeCommand` scrie SoC-ul direct din request — motorul de baterie e ocolit complet

`BatteryCommandServiceImpl.java:105`

```
battery.setSocPercent(request.targetSoc().doubleValue());
batteryRepository.save(battery);
```

Reprodus (cu authority-ul forțat, ca să trec de B3):

```
PROBE3 POST {"commandType":"BLA-BLA","targetSoc":999}
-> status 201, soc in DB -> 999.0
body {"commandType":"BLA-BLA","targetSoc":999,"status":"SUCCESS",...}
```

O baterie a ajuns la 999% încărcare, cu un tip de comandă inventat, și API-ul a răspuns `201 SUCCESS`.

DE CE: ai acum **două căi de comandă care nu se cunosc între ele**. `applyCommand()` — cea din epic-5, cu validator, randament, fizică și limite 10–90% — nu e chemată de nimeni în producție (`grep applyCommand src/main` → doar declarația și implementarea). Calea vie e `executeCommand()`, care nu cheamă validatorul, nu calculează nimic și atribuie direct valoarea primită de la client.

`status = "SUCCESS"` e hardcodat: câmpul spune „am verificat și a mers”, dar nu s-a verificat nimic. Un audit trail care raportează mereu succes nu e audit, e decor.

Și `EnergyEngine` e tot cod mort, din pass 4 — nimeni nu-l apelează.

FIX: `executeCommand` trebuie să fie *adaptorul HTTP* → *domeniu*, nu o a doua implementare. Traduce `BatteryCommandRequest` într-un `BatteryCommand` (acțiune + putere), cheamă `applyCommand`, prinde excepțiile de limită și scrie în `CommandExecution` rezultatul **real** (`SUCCESS` / `REJECTED` + motiv). Dacă modelul corect e „mergi către un SoC țintă”, atunci ăla e un al treilea tip de comandă și trebuie exprimat *prin* motor, nu în locul lui.

B5. Zero verificare de proprietar pe comenzi — comanzi și citești bateria oricui

`BatteryCommandServiceImpl.java:97–118`, `BatteryController.java:58–63`

Reprodus:

```
PROBE5 strain@test.ro a comandat casa lui owner@test.ro -> OK, executedBy=strain@test.ro, soc=80.0
PROBE9 istoricul comenzilor vazut de un strain -> status 200
body [{"houseId":2,"commandType":"CHARGE","targetSoc":70,"executedBy":"owner@test.ro",...}]
```

`executeCommand` primește `username`, dar îl folosește **doar ca etichetă** în `CommandExecution.executedBy`. Nu compară niciodată casa cu proprietarul. Iar `getCommands(houseId)` nici măcar nu primește `Principal` — nu are cum să verifice.

DE CE: autorizarea are două întrebări diferite, iar `@PreAuthorize` răspunde doar la prima. „Ai voie să dai comenzi?” e o permisiune — se rezolvă la nivel de rol, în adnotare. „Ai voie să dai comenzi **acestei case**?” e ownership — depinde de date, deci se rezolvă obligatoriu în serviciu, după ce ai încărcat entitatea. Cu doar prima verificare, orice user autentificat controlează bateria oricui, având nevoie doar de un `houseId` — care e un număr consecutiv, deci ghicibil prin numărare.

Ai deja tiparul corect scris, în `BatteryQueryServiceImpl.java:31`. Nu l-ai dus și pe calea nouă.

FIX: încarcă `House`, compară `house.getOwner().getId()` cu id-ul userului (sau lasă ADMIN să treacă), aruncă `HouseAccessDeniedHandler` altfel — și adaugă `Principal` la `getCommands`, plus un `findByHouseId` filtrat.

B6. `BatteryCommandRequest` n-are nicio validare — `@Valid` din controller nu are ce verifica

`BatteryCommandRequest.java:3–7`

```
public record BatteryCommandRequest(
    String commandType,
    Integer targetSoc
) {}
```

Reprodus, cu `targetSoc` lipsă din JSON:

```
PROBE4 {"commandType":"CHARGE"} -> status 500
      NullPointerException: Cannot invoke "java.lang.Integer.doubleValue()"
      because the return value of "BatteryCommandRequest.targetSoc()" is null
```

DE CE: `@Valid` pe `@RequestBody` pornește Bean Validation, dar Bean Validation verifică **doar constrângerile pe care le găsește**. Pe un record fără nicio adnotare nu are ce verifica, deci trece orice. Câmpul lipsă devine `null` (Jackson), ajunge nevalidat până la `.doubleValue()` și crapă acolo — adică o eroare de *input* (400, vina clientului) iese ca eroare de *server* (500, vina ta). Diferența contează: 400 îți spune clientului ce să repare, 500 îți spune să te sune pe tine.

`commandType` e `String` liber, deși ai deja enum-ul `BatteryAction` cu exact valorile permise — de-aia a trecut `"BLA-BLA"` la B4.

FIX: `@NotNull BatteryAction commandType` (enum, nu `String` — Jackson respinge singur o valoare necunoscută) și `@NotNull @Min(0) @Max(100) Integer targetSoc`.

B7. `Telemetry` nu e entitate — tot slice-ul de telemetrie e inert

`Telemetry.java:11-17`, `TelemetryRepository.java` (fișier comentat integral)

```
@Getter @Setter @Builder @AllArgsConstructor @NoArgsConstructor
@Table(name="Telemetry")          // <-- lipsește @Entity
public class Telemetry {
```

Reprodus — tabelele create efectiv de Hibernate la pornire:

```
PROBE6 tabele in H2 -> batteries command_executions houses user_permissions users
```

Nicio tabelă `telemetry`.

DE CE: `@Table` e doar un *modifier* — spune „când vei mapa clasa asta, folosește numele ăsta de tabelă”. `@Entity` e cel care o înregistrează în persistence unit. Fără el, Hibernate nici nu se uită la clasă: `@Id`, `@GeneratedValue`, `@ManyToOne`, `@JoinColumn` rămân adnotări moarte. Nu primești nicio eroare, pentru că din punctul lui de vedere clasa pur și simplu nu există.

Combinat cu `TelemetryRepository` comentat integral și cu `PvGenerator` comentat, `LoadGenerator` / `SimulationClock` clase goale — slice-ul `telemetry` e schelet comis, nu funcționalitate. Nu e greșit să comiți schelet, dar atunci spune-i pe nume în commit (**WIP: schelet telemetry**), altfel peste două săptămâni îl citești ca terminat.

FIX: `@Entity` pe clasă, decommentează repository-ul și corectează numele metodei derivate — `findByHouseIdOrderByRecordedAtDesc` cere un câmp `recordedAt` care în entitate se numește `localDateTime`; Spring Data crapă la pornire dacă numele nu se potrivește cu un câmp real.

🟡 Importante

#	Problemă	Unde
M1	Numele celor două DTO-uri de telemetrie sunt inversate. <code>TelemetryResponse</code> e de fapt pagina (<code>content</code> , <code>pageNumber</code> , <code>totalElements</code>), iar <code>HouseTelemetryPageResponse</code> e înregistrarea individuală. Mai grav, <code>TelemetryResponse</code> se conține pe sine: <code>List<TelemetryResponse></code> <code>content</code> — pagina de pagini	<code>TelemetryResponse.java:5-11</code> , <code>HouseTelemetryPageResponse.java:5-14</code>
M2	<code>CommandExecution.houseId</code> e <code>Long</code> crud, fără <code>@ManyToOne</code> spre <code>House</code> — singura entitate din proiect care nu-și leagă relația. Fără FK, poți salva comenzi pentru case inexistente	<code>CommandExecution.java:21-22</code>
M3	<code>commandType</code> și <code>status</code> sunt <code>String</code> în entitate și în DTO, deși <code>BatteryAction</code> există deja. Statusul are 2-3 valori posibile; ca <code>String</code> acceptă orice	<code>CommandExecution.java:24-31</code>
M4	<code>House house = houseRepository.findById(...)</code> — variabila e încărcată și nu se folosește niciodată . Ai vrut un check de existență; scris așa, arată ca o intenție abandonată (și e locul exact unde trebuia să stea verificarea de proprietar din B5)	<code>BatteryCommandServiceImpl.java:98-99</code>
M5	<code>HouseAccessDeniedHandler</code> — excepție de business numită „...Handler” (numele sugerează un <code>AccessDeniedHandler</code> de Spring Security, care e cu totul altceva) și nemapată în <code>GlobalExceptionHandler</code> → refuzul de acces va ieși ca 500, nu 403. Azi nu se vede pentru că B2 crapă înainte	<code>HouseAccessDeniedHandler.java:5</code> , <code>GlobalExceptionHandler.java:16-42</code>
M6	<code>getCommandsForHouse</code> nu e paginat și nu e sortat — <code>findByHouseId</code>	<code>CommandExecutionRepository.java:9</code>

#	Problemă	Unde
	întoarce tot istoricul, în ordinea bazei de date. La un simulator care scrie o comandă la fiecare tick, lista crește nelimitat	
M7	<code>BatteryCommandService</code> amestecă acum trei niveluri în aceeași interfață: <code>applyCommand(Battery, ...)</code> (domeniu, primește entitate), <code>executeCommand(Long, DTO, String)</code> (aplicație, primește DTO) și <code>getCommandsForHouse</code> (citire — ar trebui în query service, ai deja <code>BatteryQueryService</code>)	<code>BatteryCommandService.java:8–13</code>
M8	<code>createdAt</code> se setează manual cu <code>LocalDateTime.now()</code> , deși ai deja <code>AbstractAuditable</code> în proiect pentru exact asta	<code>CommandExecution.java:36–37</code> , <code>AbstractAuditable.java</code>

Restanțe din pass 4 — verificate pe cc74f0c

#	Stare	Dovadă
B1 — <code>maven-failsafe-plugin</code> lipsă, cele 3 clase <code>*IT</code> nu rulează	deschis	<code>pom.xml</code> <code><build><plugins></code> = spring-boot + compiler + jacoco. Momentan irelevant: B1 runda 5 oprește tot
B2 — motorul fără apelanți	deschis, mutat	endpoint-ul există acum, dar cheamă <code>executeCommand</code> , nu <code>applyCommand</code> (B4 runda 5)
B3 — clamp SoC vs excepție, două politici contradictorii	deschis	<code>BatteryCommandServiceImpl.java:76–78,86–88</code> vs <code>BatteryCommandValidatorImpl.java:41–43,55–57</code> — neatins
B4 — <code>@NotNull</code> pe <code>double</code> primitiv	deschis	<code>BatteryCommand.java:9–10</code> — neatins
M1/M2 — <code>DEFAULT_CAPACITY_KWH</code> hardcodat și duplicat	deschis	<code>BatteryCommandServiceImpl.java:23–25</code> și <code>BatteryCommandValidatorImpl.java:15–17</code> — aceleași 3 constante, în două clase
M7 — <code>BatteryPowerLimitException</code> / <code>BatterySocLimitException</code> nemapate	deschis	<code>GlobalExceptionHandler.java:16–42</code>
M9 — <code>EnergyCommandTests</code> duplicat al lui <code>EnergyEngineTests</code>	deschis	ambele fișiere există, identice
M11 — <code>@Component</code> în loc de <code>@Service</code>	deschis	<code>BatteryCommandServiceImpl.java:20</code> , <code>BatteryCommandValidatorImpl.java:11</code> , <code>BatteryQueryServiceImpl.java:15</code>
C1 — lipsă newline la final de fișier	deschis, extins	6 fișiere din commit-urile noi: <code>CommandExecution</code> , <code>BatteryController</code> , <code>CommandExecutionMapper</code> , <code>BatteryCommandServiceImpl</code> , <code>UserPermissions</code> , <code>User</code>
C2 — <code>public class</code> pe linie separată	deschis	<code>BatteryCommandValidatorImpl.java:12–13</code>
C3 — <code>BATTERY_POWER_LIMIT = "BATTERY_POWER_LIMIT"</code>	deschis	<code>ErrorConstants.java:11–12</code>

Restanțe din pass 3

#	Stare	Dovadă
B1 — oricine se înregistrează ADMIN	● deschis	<code>AuthRegisterRequest.java:11-12</code> primește <code>userType</code> de la client → <code>AuthServiceImpl.java:55,58</code> . Am creat un ADMIN cu un simplu POST public în probe
B2 — slice-ul battery inaccesibil	● parțial	<code>BATTERY_VIEW</code> ✅ reparat; <code>BATTERY_CONFIG</code> / <code>BATTERY_COMMAND</code> ● (B3 runda 5)
B3 — handler cu body <code>null</code> și status intern greșit	● deschis	<code>GlobalExceptionHandler.java:24-26</code> : <code>.status(NO_CONTENT)</code> pe un 404, iar <code>.error()</code> nu e setat niciodată → body-ul răspunsului e <code>null</code>
B4 — <code>UserController./add</code> mort	● deschis	<code>UserController.java:28</code> cere <code>hasAuthority('ADMIN')</code> , dar <code>ADMIN</code> e <code>UserType</code> , nu permisiune — nu apare în niciun token. În plus <code>userResponse</code> se calculează și se aruncă (<code>:30-31</code>), răspunsul are body gol
B5 — <code>ddl-auto</code> / Flyway	▮ lăsat deoparte	migrările nu sunt încă predate; se reia când ajungem la ele

🟢 Cleanups

- **C4 — Importuri nefolosite:** `OffsetDateTime` în `CommandExecution.java:7` , `Instant` în `ApiErrorResponse.java:5` , `Instant` / `LocalDateTime` parțial în `AuthServiceImpl.java:19-20` .
- **C5 — `efficiencyPercent`** (`Battery.java:32`) — typo încă prezent în identificator, deși `toString()` afișează `efficiencyPercent` . Codul și log-ul spun lucruri diferite.
- **C6 — `ComunityStateResponse`** — „Comunity” → `CommunityStateResponse` . Cât timp nimic nu-l folosește încă, redenumirea e gratis.
- **C7 — `Telemetry.toString()` și `equals()` includ `house`** (`Telemetry.java:50,58`) — pe o relație `LAZY` , un `toString()` într-un log poate declanșa aceeași `LazyInitializationException` din B2. Regula: `toString` / `equals` pe entități nu ating niciodată relații.
- **C8 — mesaje de commit:** `commit` , `COMMIT` , `battery` , `code review` . Din `git log` nu se poate reconstitui nimic. Un commit = o propoziție la imperativ: `add battery command endpoint` , `persist command executions` .
- **C9 — igienă de branch-uri:** `master` , `epic-6` și `feat/epic-5` sunt același fir; `feat/fundatie` e vechi de o lună și complet inclus în `master` . Șterge branch-urile terminate, ține unul singur de lucru — altfel nu mai știi (și nu mai știi nici

eu) unde e adevărul.

Before / After (criticele)

B1 — testul nu se mai rupe la fiecare dependență nouă

```
// Before
@Mock private BatteryRepository repository;
@Mock private BatteryCommandValidator validator;
private BatteryCommandService service;

@BeforeEach
void setup() {
    service = new BatteryCommandServiceImpl(repository, validator); // se rupe la orice dependență nouă
}

// After
@Mock private BatteryRepository repository;
@Mock private BatteryCommandValidator validator;
@Mock private CommandExecutionRepository commandExecutionRepository;
@Mock private HouseRepository houseRepository;
@InjectMocks private BatteryCommandServiceImpl service; // Mockito completează singur
```

B2 — ownership-ul citit fără să depindă de sesiunea Hibernate

```
// Before – BatteryQueryServiceImpl
public BatteryResponse getBatteryByHouseId(Long houseId, String email) {
    Battery battery = batteryRepository.findBatteryByHouseId(houseId).orElseThrow(...);
    if (user.getUserType() != UserType.ADMIN
        && !battery.getHouse().getOwner().getId().equals(user.getId())) { // proxy fără sesiune -> 500
        // ...
    }
}

// After
@Transactional(readOnly = true)
public BatteryResponse getBatteryByHouseId(Long houseId, String email) { ... }

// sau, mai bine, în BatteryRepository:
@Query("select b from Battery b join fetch b.house h join fetch h.owner where h.id = :houseId")
Optional<Battery> findByIdWithOwner(@Param("houseId") Long houseId);
```

B3 — permisiunile cerute trebuie să fie permisiuni acordate

```
// Before – AuthServiceImpl.permissionsForType()
permissions.addAll(Set.of(
    UserPermissions.HOUSE_VIEW, UserPermissions.HOUSE_MANAGE,
    UserPermissions.BATTERY_VIEW, UserPermissions.BATTERY_MANAGE)); // MANAGE nu e cerut nicăieri
                                                                    // CONFIG și COMMAND nu se acordă nicăieri

// After
permissions.addAll(Set.of(
    UserPermissions.HOUSE_VIEW, UserPermissions.HOUSE_MANAGE,
    UserPermissions.BATTERY_VIEW, UserPermissions.BATTERY_CONFIG,
    UserPermissions.BATTERY_COMMAND));
// + șterge BATTERY_MANAGE din enum dacă rămâne necerut
```

B4 — endpoint-ul cheamă motorul, nu îl ocolește

```
// Before – BatteryCommandServiceImpl.executeCommand
battery.setSocPercent(request.targetSoc().doubleValue()); // fără validare, fără fizică
batteryRepository.save(battery);
CommandExecution execution = CommandExecution.builder()
    .status("SUCCESS") // mereu SUCCESS
    ...

// After
BatteryCommand command = new BatteryCommand(request.commandType(), request.powerKw());
String status;
try {
    applyCommand(battery, command, DEFAULT_TICK_HOURS); // validator + randament + limite
    status = "SUCCESS";
} catch (BatteryPowerLimitException | BatterySocLimitException e) {
    status = "REJECTED"; // auditul spune adevărul
}
```

B5 — ownership pe calea de comandă

```
// After – în executeCommand, imediat după ce ai încărcat casa
User user = userRepository.findByEmail(username).orElseThrow(UserNotFoundException::new);
if (user.getUserType() != UserType.ADMIN && !house.getOwner().getId().equals(user.getId())) {
    throw new HouseAccessDeniedHandler();
}
// + aceeași verificare în getCommandsForHouse(Long houseId, String username)
```

B6 — DTO care se validează singur

```
// Before
public record BatteryCommandRequest(String commandType, Integer targetSoc) {}

// After
public record BatteryCommandRequest(
    @NotNull(message = "Command type is required")
    BatteryAction commandType, // enum: Jackson respinge singur o valoare necunoscută
    @NotNull(message = "Target SoC is required")
    @Min(value = 0) @Max(value = 100)
    Integer targetSoc
) {}
```

B7 — clasa devine entitate

```
// Before
@Table(name = "Telemetry")
public class Telemetry { ... }

// After
@Entity
@Table(name = "telemetry")
public class Telemetry { ... }
```

Ordine recomandată de reparare

1. **B1** — o linie. Până nu compilează testele, nimic din ce urmează nu poate fi dovedit.
 2. **B3** — două valori în `permissionsForType()`. Fără ele, endpoint-ul tău nou nu poate fi atins nici măcar manual din Swagger.
 3. **B2** — `@Transactional(readOnly = true)`. Repară singurul endpoint de citire al slice-ului.
 4. **B6** — validări pe DTO. leftin și taie jumătate din inputurile absurde.
 5. **B5** — ownership pe `executeCommand` și `getCommands`. E securitate, nu cosmetică.
 6. **B4** — leagă `executeCommand` de `applyCommand`. Aici trebuie mai întâi să închizi B3 din pass 4 (clamp SAU reject), altfel legi un motor cu două politici contradictorii.
 7. **B7** — `@Entity` + repository decommentat, ca slice-ul de telemetrie să pornească pe picioare drepte.
 8. **Un test MockMvc pentru fiecare endpoint** — 200 pe calea fericită, 403 pentru un user fără permisiune, 403 pentru casa altcuiva, 400 pe input invalid. Patru teste care ar fi prins B2, B3, B5 și B6 simultan.
 9. Apoi restanțele: failsafe în `pom.xml`, `capacityKwh` în entitate, excepțiile mapate.
-

Q&A — verifică-ți înțelegerea

1. `GET /battery` crapă cu `LazyInitializationException` pentru un USER, dar merge pentru un ADMIN — cu **exact același** cod și aceeași bază de date. Explică de ce, pornind de la linia `if (user.getUserType() != UserType.ADMIN && !battery.getHouse()...)`. Ce ai fi văzut dacă puneai `open-in-view: true` în `application.yml` — și de ce n-ar fi fost o reparație, ci o mușamalizare?
2. Un client trimite `{"commandType":"BLA-BLA","targetSoc":999}` și primește `201 SUCCESS`, cu SoC-ul salvat la 999%. Enumeră **toate** apărările care ar fi trebuit să oprească cererea asta, în ordinea în care ar fi trebuit să se declanșeze, și spune pentru fiecare de ce n-a existat sau n-a fost chemată.
3. `BATTERY_VIEW` funcționează, `BATTERY_COMMAND` dă 403 — ambele sunt valori în același enum `UserPermissions`. Care e, exact, diferența dintre „o permisiune există” și „un user o are”? Scrie testul (o singură metodă `MockMvc`) care ar fi prins bug-ul ăsta la toate cele trei runde în care a apărut.

(răspunde-le în scris sau la următoarea sesiune, apoi spune „next”)

Runda 4 — 2026-08-19 — sha `e21d7c9`

Motorul de baterie (`applyCommand` + `BatteryCommandValidator`) și bilanțul energetic (`EnergyEngine`). Fizica și separarea validator/serviciu — corecte. Tipar: *scrie motorul, nu închide slice-ul*.

- B1 — `maven-failsafe-plugin` lipsă → 16 teste `*IT` invizibile, build verde peste ele. **NEATINS**
- B2 — toată logica din epic-5 fără apelanți. **Mutat** — endpoint-ul există în runda 5, dar ocolește motorul
- B3 — clamp SoC inaccesibil: validatorul refuză, serviciul limitează — două politici contradictorii. **NEATINS**
- B4 — `@NotNull` pe `double` primitiv; putere negativă = no-op tăcut. **NEATINS**
- M1-M11 — `capacityKwh` lipsă din entitate, constante duplicate, convenție de semn nedocumentată, limite de racord ignorate, excepții nemapate, `EnergyCommandTests` duplicat, `@Component` în loc de `@Service`. **NEATINSE**
- C1-C5 — newline final, formatare, `ErrorConstants`, `efficiencyPercent`. **NEATINSE**

Runda 3 — 2026-08-?? — fundația (auth / house / battery)

- B1 — oricine se poate înregistra ca ADMIN (`userType` vine de la client). **NEATINS**
- B2 — slice-ul `battery` = 403 pentru toți (`battery:view` vs `BATTERY_VIEW`).  **parțial închis** — `BATTERY_VIEW` reparat; `BATTERY_CONFIG` / `BATTERY_COMMAND` redeschis ca B3 runda 5
- B3 — `GlobalExceptionHandler` cu body `null` și `.status(NO_CONTENT)` pe 404. **NEATINS**
- B4 — `UserController.add` mort (`hasAuthority('ADMIN')` — `ADMIN` nu e permisiune). **NEATINS**
- B5 — `ddl-auto` / Flyway. **amânat** — migrările nu sunt încă predate

